

[Docs](#) > [Routing](#) > [Setup](#)

Setup locale-based routing

Prefer to watch a video?

 **Routing setup**

In order to use unique pathnames for every language that your app supports, `next-intl` can be used to handle the following routing setups:

1. Prefix-based routing (e.g. `/en/about`)
2. Domain-based routing (e.g. `en.example.com/about`)

In either case, `next-intl` integrates with the App Router by using a top-level `[locale]` [dynamic segment](#) [↗] that can be used to provide content in different languages.

Initial setup

To get started with locale-based routing, we'll set up the following files:

1 `src/i18n/routing.ts`

We'll use `routing.ts` as a central place to define our routing configuration:

```
src/i18n/routing.ts

import {defineRouting} from 'next-intl/routing';

export const routing = defineRouting({
  // A list of all locales that are supported
  locales: ['en', 'de'],

  // Used when no locale matches
  defaultLocale: 'en'
});
```

Depending on your requirements, you may wish to customize your routing configuration later—but let's finish with the setup first.

2

src/proxy.ts

Once we have our routing configuration in place, we can use it to set up a proxy:

src/proxy.ts

```
import createMiddleware from 'next-intl/middleware';
import {routing} from './i18n/routing';

export default createMiddleware(routing);

export const config = {
  // Match all pathnames except for
  // - ... if they start with `/api`, `/trpc`, `/_next` or `/_vercel`
  // - ... the ones containing a dot (e.g. `favicon.ico`)
  matcher: '/((?!api|trpc|_next|_vercel|.*\\..*).*)'
};
```

Note: `proxy.ts` was called `middleware.ts` up until Next.js 16.

> How can I match pathnames that contain dots like `/users/jane.doe` ?

3

src/i18n/navigation.ts

Additionally, we can use our routing configuration to set up the navigation APIs:

src/i18n/navigation.ts

```
import {createNavigation} from 'next-intl/navigation';
import {routing} from './routing';

// Lightweight wrappers around Next.js' navigation
// APIs that consider the routing configuration
export const {Link, redirect, usePathname, useRouter, getPathname} =
  createNavigation(routing);
```

4

src/i18n/request.ts

Now, we can read the matched locale in our request configuration:

src/i18n/request.ts

```
import {getRequestConfig} from 'next-intl/server';
import {hasLocale} from 'next-intl';
import {routing} from './routing';

export default getRequestConfig(async ({requestLocale}) => {
  // Typically corresponds to the `[locale]` segment
  const requested = await requestLocale;
  const locale = hasLocale(routing.locales, requested)
    ? requested
    : routing.defaultLocale;

  return {
    locale
    // ...
  };
});
```

5

src/app/[locale]/layout.tsx

To complete our setup, we'll move all of our existing layouts and pages into the `[locale]` segment:

```
src
├── app
│   └── [locale]
│       ├── layout.tsx
│       ├── page.tsx
│       └── ...
```

The `locale` that was matched is now available via the `[locale]` param:

app/[locale]/layout.tsx

```
import {NextIntlClientProvider, hasLocale} from 'next-intl';
import {notFound} from 'next/navigation';
import {routing} from '@/i18n/routing';

type Props = {
  children: React.ReactNode;
  params: Promise<{locale: string}>;
};

export default async function LocaleLayout({children, params}: Props) {
  // Ensure that the incoming `locale` is valid
  const {locale} = await params;
  if (!hasLocale(routing.locales, locale)) {
```

```
    notFound();  
  }  
  
  // ...  
}
```

That's all it takes! From here, you can [configure your routing](#) to cater to your specific needs.

In case you ran into an issue, have a look at the [App Router example](#) to explore a working app.

Static rendering

When using locale-based routing, `next-intl` will currently opt into dynamic rendering when APIs like `useTranslations` are used in Server Components. This is a limitation that we aim to remove in the future, but as a stopgap solution, `next-intl` provides a temporary API that can be used to enable static rendering.

1 Add `generateStaticParams`

Since we are using a dynamic route segment for the `[locale]` param, we need to use [generateStaticParams](#) ↗ so that the routes can be rendered at build time.

Depending on your needs, you can add `generateStaticParams` either to a layout or pages:

1. **Layout:** Enables static rendering for all pages within this layout (e.g. `app/[locale]/layout.tsx`)
2. **Individual pages:** Enables static rendering for a specific page (e.g. `app/[locale]/page.tsx`)

Example:

```
import {routing} from '@/i18n/routing';

export function generateStaticParams() {
  return routing.locales.map((locale) => ({locale}));
}
```

▼ Do I need to return all locales from `generateStaticParams` ?

If you prefer to only render certain locales at build time, or none at all, you can be selective about the ones you return from `generateStaticParams` (see [Static Rendering](#) ↗ in the Next.js docs).

2 Add `setRequestLocale` to all relevant layouts and pages

`next-intl` provides an API that can be used to distribute the locale that is received via `params` in layouts and pages for usage in all Server Components that are rendered as part of the request.

`app/[locale]/layout.tsx`

```
import {setRequestLocale} from 'next-intl/server';
import {hasLocale} from 'next-intl';
import {notFound} from 'next/navigation';
import {routing} from '@/i18n/routing';

type Props = {
  children: React.ReactNode;
  params: Promise<{locale: string}>;
};

export default async function LocaleLayout({children, params}: Props) {
  const {locale} = await params;
  if (!hasLocale(routing.locales, locale)) {
    notFound();
  }

  // Enable static rendering
  setRequestLocale(locale);

  return (
    // ...
  );
}
```

`app/[locale]/page.tsx`

```
import {use} from 'react';
import {setRequestLocale} from 'next-intl/server';
import {useTranslations} from 'next-intl';

export default function IndexPage({params}) {
  const {locale} = use(params);

  // Enable static rendering
  setRequestLocale(locale);

  // Once the request locale is set, you
  // can call hooks from `next-intl`
  const t = useTranslations('IndexPage');

  return (
    // ...
  );
}
```

Keep in mind that:

1. The locale that you pass to `setRequestLocale` should be validated (e.g. in your [root layout](#)).
2. You need to call this function in every page and every layout that you intend to enable static rendering for since Next.js can render layouts and pages independently.
3. `setRequestLocale` needs to be called before you invoke any functions from `next-intl` like `useTranslations` or `getMessages`.

▼ How does `setRequestLocale` work?

`next-intl` uses `cache()` [↗](#) to create a mutable store that holds the current locale. By calling `setRequestLocale`, the current locale will be written to the store, making it available to all APIs that require the locale.

Note that the store is scoped to a request and therefore doesn't affect other requests that might be handled in parallel while a given request resolves asynchronously.

▼ Why is this API necessary?

Next.js currently doesn't provide an API to read route params like `locale` at arbitrary places in Server Components (see [vercel/next.js #58862](https://vercel.com/blog/next.js-v12-0-0#internationalization)). The `locale` is fundamental to all APIs provided by `next-intl`, therefore passing this as a prop throughout the tree doesn't stand out as particularly ergonomic.

Due to this, `next-intl` uses its middleware to attach an `x-next-intl-locale` header to the incoming request, holding the negotiated locale as a value. This technique allows the locale to be read at arbitrary places via `headers().get('x-next-intl-locale')`.

However, the usage of `headers` opts the route into dynamic rendering.

By using `setRequestLocale`, you can provide the locale that is received in layouts and pages via `params` to `next-intl`. All APIs from `next-intl` can now read from this value instead of the header, enabling static rendering.

3 Use the `locale` param in metadata

In addition to the rendering of your pages, also page metadata needs to qualify for static rendering.

To achieve this, you can forward the `locale` that you receive from Next.js via `params` to the [awaitable functions from next-intl](#).

page.tsx

```
import {getTranslations} from 'next-intl/server';

export async function generateMetadata({params}) {
  const {locale} = await params;
  const t = await getTranslations({locale, namespace: 'Metadata'});

  return {
    title: t('title')
  };
}
```



[Docs](#) · [Learn](#) · [Examples](#) · [Blog](#) · [v4](#)

[X](#) · [Bluesky](#) · [GitHub](#) · Hosted on  **Vercel**